

## Part 1: The Phantom Bug

- Write a program that tosses 50,000,000 random  $(x, y)$  points into a  $1 \times 1$  square to approximate  $\pi$  ( $\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$ ).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.
- **Deliverable:** Run it 5 times. Record the value of  $\pi$ .
- *The Lesson:* The code runs fast, but gives  $\pi \approx 1.8$  to  $2.4$  because threads overwrite each other's memory writes (data race).

## Part 2: The Synchronization Trap

- Fix the race condition by adding `synchronized` to the increment or using `AtomicLong.incrementAndGet()`.
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate ( $\approx 3.1415$ ), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

## Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for  $T \in \{1, 2, 4, 8, 16, 32\}$  threads

on their machine with \$100,000,000\$ iterations.

- Fill out this grid:

| Threads<br>(T) | Runtime<br>(ms)   | Speedup vs. 1 Thread<br>( $T/N T_1$ ) | Efficiency<br>( $T \text{Speedup}$ ) |
|----------------|-------------------|---------------------------------------|--------------------------------------|
| 1              | 388<br>(baseline) | 1.0x                                  | 100%                                 |
| 2              | 200               | 1.94x                                 | 97.0%                                |
| 4              | 128               | 3.03x                                 | 75.8%                                |
| 8              | 87                | 4.46x                                 | 55.7%                                |
| 16             | 86                | 4.51x                                 | 28.2%                                |

### Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

**My measurement:  $T = 8$  took 87 ms,  $T = 16$  took 86 ms — a 1.2% improvement for double the threads.**

First, a correction to the premise: my machine is not an 8-core CPU. It is an Intel Core i7-12700H with **14 physical cores (6 P-cores + 8 E-cores) and 20 logical threads**. The answer is the same shape regardless — the machine had already exhausted its useful capacity well before  $T = 16$ .

**Threads are not execution units.** A thread is a scheduling abstraction; a core is what actually executes instructions. Once  $T$  exceeds the number of cores that can usefully run the work, extra threads add no capacity — the OS just time-slices the same silicon among more runnable entities. Speedup is bounded by  $\min(T, \text{usable cores})$ , never by  $T$ .

**The efficiency column shows the saturation directly:** 97.0% at  $T=2$ , 75.8% at  $T=4$ , 55.7% at  $T=8$ , then 28.2% at  $T=16$ . Each doubling past  $T=8$  halves efficiency while runtime stays flat — the signature of a saturated resource. The work isn't going faster; it's just being divided among more threads.

**Peak speedup was only 4.51×, far short of 14, for three hardware reasons:**

- **E-cores are much slower than P-cores.** Work is split into equal chunks, but elapsed time is set by the slowest thread. Once threads land on E-cores, those chunks become the critical path and everyone waits. Equal partitioning is the wrong policy on a heterogeneous CPU.
- **All-core turbo is far below single-core turbo.** At  $T=1$  one P-core boosts to its maximum; with every core loaded the package hits power and thermal limits and clocks drop. Per-core throughput fell from 258,000 points/ms at  $T=1$  to 144,000 at  $T=8$  — just 56% of the unencumbered rate. The baseline is measured under the most favourable conditions the chip ever offers, which depresses every speedup computed against it.
- **SMT threads share execution resources rather than adding them.** Hyper-Threading duplicates registers, not the execution back-end. This kernel is tight register-resident arithmetic with no stalls, so one thread already saturates its core and the sibling finds no idle issue slots.

**Beyond saturation, more threads are pure cost.** Total work is fixed, so per-thread work shrinks as  $1/T$  while thread creation and scheduler pressure grow as  $O(T)$ . Past the core count the scheduler must preempt, and each context switch costs 1–10  $\mu\text{s}$  directly plus a much larger indirect penalty as the evicted thread loses its cache working set.

**Conclusion: the optimum here is  $T = 8$ , not 16 or 32.** Matching thread count to logical processor count is a common and costly mistake.

2. Why was the synchronized version in Part 2 slower than running on one single core?

Measured: 2155 ms with 4 threads versus 199 ms single-threaded — 10.83× slower.

The monitor serialises every increment, so all ~39.3 million happen one at a time regardless of thread count. The parallel fraction is effectively zero — the threads are just a queue. That explains why it isn't faster; it's slower because multithreading adds costs the serial version never pays:

- **Lock acquire/release ~39 million times.** Under contention HotSpot inflates the monitor to OS-backed park/unpark. Measured cost: 49.8 ns per increment, against ~1 ns for an L1 hit.
- **Cache-line ping-pong — the dominant cost.** `synchronHits` sits in a 64-byte line that must be held in Modified state by whichever core is incrementing. Each handover invalidates it elsewhere and transfers it across the interconnect: tens of cycles versus ~4 for an L1 hit. The single-threaded version keeps its counter in a CPU register — zero memory and coherence traffic.
- **Convoying.** If the OS deschedules the lock holder, every waiter stalls.

`AtomicLong` at 606 ms is 3.56× faster than `synchronized` because CAS stays in user space (10.4 ns/increment), but it still takes exclusive ownership of that cache line every time, so it remains 3.05× slower than not sharing at all. No lock design beats the interconnect.

Part 3 proves the point: removing the shared variable gives 100M points in 87 ms — roughly 50× faster than the synchronized version.

Upload your solutions before the class end-time to the Git repo that you created for benchmarks. Share repo access with @sufyanism (GitHub/GitLab).